

Mock exam — TEST Exam v2 (Jun 3, 2026)

The format changed. Read this first.

#	Exercise	Pts	Type	In this project
1	Acceptance tests (Gherkin)	5	text	<code>ise.acceptance.exam_login</code> + §1 below
2	Requirements elicitation	8	text	§2 below
3	Mocking — SmartHome	25	code	<code>ise.mocking.exam_smarthome</code>
4	Black-box testing	10	text	<code>ise.blackbox.exam_gamelauncher</code> + §4 below
5	SOLID — course enrollment	25	code	<code>ise.solid.exam_enrollment</code>
6	UML modeling	7	diagram	§6 below
7	Quiz	20	MCQ	§7 below

What's different from your two real exams

- **There is no REST exercise at all.** The 20–22 points that used to be REST are now SOLID (25).
- **Mocking is worth 25**, up from 15/18 — and the collaborator methods are `void`, so `verify()` carries the whole test. There is no return value to fall back on.
- **Black-box testing is now its own exercise**, worth 10, answered as text tables rather than as JUnit code.
- **Two new text exercises:** Gherkin acceptance tests and requirements elicitation, 13 points combined, needing no code at all.
- Only **57 minutes** of working time, versus 1h40 before.

The REST work in this project is still worth keeping — it was 20–22 points on both real exams — but if this mock reflects the next paper, **SOLID and the SmartHome mocking task are where the marks are.**

§1 — Acceptance tests: the two Gherkin scenarios

Feature: Log in to a personal account

Scenario: Successful login with valid credentials

Given I am on the login page
And I am registered on the website with username "student" and password "password123"
When I enter the username "student"
And I enter the password "password123"
And I log in on the website
Then I should access my account
And I should not see any error message

Scenario: Login refused when invalid credentials are provided

Given I am on the login page
And I am registered on the website with username "student" and password "password123"
When I enter the username "student"
And I enter the password "wrongpassword"
And I log in on the website
Then I should not access my account
And I should see an error message
And I should remain on the login page

Both satisfy the two graded constraints: ≥ 2 **Given steps** and ≥ 2 **Then steps**.

What was weak in the submitted answer

- and i enter the password "assword123"and log in on the website — **two steps merged onto one line**, with no space. A Gherkin parser reads this as a single step and cannot match it. This alone can cost the scenario.
- The invalid-credentials scenario changed **both** the username ("chud") and the password ("assword123"). Change one variable at a time, or you cannot tell which one caused the refusal. Two separate scenarios would be better still.
- then i should not be able to access my account / and i should see a error message — only 2 Then steps, which meets the minimum, but "*remain on the login page*" is explicitly in the problem statement and is free marks.
- Lowercase keywords (given, when, then) are legal Gherkin and are not an error — but capitalising them makes the structure obvious to a human grader in a hurry.

Executable version: [LoginAcceptanceTest.java](#) — each Gherkin line appears as a comment above the code that realises it, and Given = Arrange, When = Act, Then = Assert.

§2 — Requirements elicitation: the training tracker

1. Three stakeholder groups

Stakeholder	Goal or need
Head of the club	Training organised by sport type, team and individual player, to get an overview across the whole club
Trainers	See each player's sessions (name, exercise type, duration, optional intensity) and set a maximum training time and a number of rest days per week
Players	Record a session with minimum effort — either manually or by linking a smart watch / phone app — and control who can see it

2. Four points needing clarification, and who to ask

#	What needs clarifying	Ask
1	What exactly counts as "intensity"? Which metrics (heart rate, pace, both, others), which units, and is it optional for every exercise type?	Trainers
2	Who may see a session marked <i>private</i> — only the player, or the trainer too? The scenario says "private or visible to teammates" and never says where the trainer sits.	Head of the club and trainers
3	How is the rest-day rule counted? Which day does the week start on, does a rest day mean <i>zero</i> sessions, and what happens if the player logs a session retrospectively?	Trainers
4	Which external devices must be supported, and what happens on a sync conflict (device data disagreeing with a manual entry, or a duplicate session)?	Players and the head of the club

Others worth naming if you have time: is the maximum training time per session or per day; can a player edit or delete a saved session; does the warning block saving or merely inform.

3. One possible conflict

Players want sessions to be **private**; trainers want to **see every session** of their players, and the system must **notify the trainer** when no rest days were taken. Those requirements are in direct opposition: honouring privacy breaks the trainer's oversight, and honouring oversight makes "private" meaningless. It has to be resolved as policy — e.g. private hides the *detail* but the rest-day warning still fires with no session data attached.

§4 — Black-box testing: Game Launcher Beta Access

Verified by `GameLauncherBlackBoxTest.java` — every row below is executed as a real assertion.

Part 1 — equivalence classes

`accountType`

ATC1	Valid	"standard"
ATC2	Valid	"vip"
ATC3	Invalid	any other value, e.g. "admin", "VIP", "", null

`characterLevel` — split per bracket, which is how you get to exactly eight classes:

CLC1	Invalid	<code>characterLevel <= 0</code>	e.g. 0, -5
CLC2	Valid	standard, level 1..19	below the standard bracket
CLC3	Valid	standard, level 20..50	inside the standard bracket
CLC4	Valid	standard, level 51..100	above the standard bracket
CLC5	Valid	vip, level 1..9	below the vip bracket
CLC6	Valid	vip, level 10..80	inside the vip bracket
CLC7	Valid	vip, level 81..100	above the vip bracket
CLC8	Invalid	<code>characterLevel >= 101</code>	e.g. 101, 150

"Valid" here means *a valid input the system accepts and evaluates*; it does not mean access is granted. CLC2 is a valid level that produces rejected.

Part 2 — the 9 representative test cases

TC1	standard	10	rejected
TC2	standard	35	accepted
TC3	standard	75	rejected
TC4	vip	5	rejected
TC5	vip	45	accepted
TC6	vip	90	rejected
TC7	standard	0	invalid
TC8	vip	101	invalid
TC9	admin	35	invalid

Coverage: TC1-TC3 cover ATC1 + CLC2/3/4, TC4-TC6 cover ATC2 + CLC5/6/7, TC7 covers CLC1, TC8 covers CLC8, TC9 covers ATC3. All eleven classes, nine cases.

Part 3 — boundary values (three-point rule)

VIP lower bracket limit = **10**, with `accountType = vip`:

TC10	9	rejected
TC11	10	accepted
TC12	11	accepted

Standard upper bracket limit = **50**, with `accountType = standard`:

TC13	49	accepted
TC14	50	accepted
TC15	51	rejected

TC11 and TC14 are the ones that earn the marks. They sit exactly *on* an inclusive boundary, and they are the only cases that fail when `>=` is written as `>`. The drill proves it: mutation *"off-by-one at*

the *VIP lower limit*" is caught by TC11 and by nothing else.

§6 — UML modeling (7 pts)

No diagram was in the PDF for this exercise, so there is nothing specific to reproduce. From the retake feedback, the two things that cost you marks before:

- "There are no numbers in the messages." — a **communication diagram** must number every message (1:, 2:, 2.1: ...). Without the numbering the sequence is undefined and the grader marked it "not a functional diagram", worth zero.
- "Incorrect representation of the object/actor. Missing name or type." — object nodes are written **name:Type** and **underlined**. Student:Andrei is backwards; it should be andrei:Student.

And from the patterns exercise: "we cannot provide points to independent elements if the model does not accomplish the pattern" — a diagram containing the right classes but not wired into the pattern scores **0**, not partial credit. Draw the pattern skeleton first (Subject with attach/detach/notify, Observer with update), then map the domain onto it.

§7 — Quiz answer key

#	Question	Answer	Why
1	Big-Bang integration requires test doubles for all components	False	Big-Bang integrates everything at once, so there is nothing to stub. It is <i>incremental</i> integration (top-down especially) that needs stubs.
2	Continuous Delivery vs Deployment	Delivery deploys to production manually; Deployment deploys automatically	Both automate the pipeline up to a release candidate. Only Deployment pushes to production with no human gate.
3	CI pipeline ordering	"Fail fast": cheap, fast checks should catch problems before expensive, slow stages run	Static analysis takes seconds; system tests take minutes.
4	Cyclomatic Complexity correlates strongly with understandability	False	It correlates strongly with <i>size</i> ; the evidence for understandability is weak. This is a lecture-specific point.
5	Compile, test and coverage in one command	gradlew build	build = assemble + check. assemble skips tests; run and javadoc do something else entirely.
6	After requirements specification comes...	Architecture design	Requirements → design → implementation → testing.
7	Design for Marcus (ADHD persona)	One-Click Quick Start widget and Mute All Reminders + passive agenda	Both reduce friction. Gamification and a social feed as home screen are exactly what the persona says derails them.
8	Requirements validation methods	Stakeholder reviews, Gap analysis	Refactoring and source-code optimisation happen after requirements, on code.
9	A use case always includes	At least one actor and a goal	Class diagrams, schemas and code are not part of a use case.

#	Question	Answer	Why
10	Too large for one sprint	Epic	Epic → Story → Task. A story is by definition sprintable.
11	Four-Phase Test order	Setup → Execute → Verify → Teardown	Same four phases as EasyMock's record → replay → exercise → verify, and as Arrange/Act/Assert plus cleanup.
12	Integration test	Tests whether several components work together correctly	One method in isolation is a unit test.
13	Usability testing data sources	Video recordings, eye tracking, log files, observations during task execution	Everything except the randomly generated unit-test stubs.
14	System test environment	A test environment simulating the customer's production environment, testing both functional and non-functional requirements	The point of a system test is realism <i>and</i> NFRs. Never in real production.
15	Test double matcher	see the table below	
16	Recognised categories of documented solutions	Design patterns, Anti-patterns	"Debugging patterns" and "execution patterns" are not categories from the lecture.
17	Changed but not staged	Modified	Git states: modified → staged → committed.
18	Units done with stubs, now combining components	Integration test	Interface errors between components is the definition.
19	Every outcome of every <code>if</code> , both true and false	Branch coverage	Statement coverage can hit 100% without ever taking a false branch.
20	The system crashes and fails to provide the service	Failure	Mistake (human) → Fault/defect (in the code) → Error (wrong internal state) → Failure (observable). The question asks for the <i>observable event</i> .

Q15 — the test double table

Double	Characteristic
Dummy	Never actually called, only satisfies the API / constructor signature
Stub	Provides fixed or configurable return values and does not verify usage
Spy	Records calls / data sent to dependencies so the test can verify them later
Mock	Expectations set in advance, later verified after the exercise stage
Fake	Real but simplified implementation (e.g. in-memory database)

All five are demonstrated as running code in [ise.testing.s18_testdoubles](#), against one SUT, so the differences are concrete rather than memorised.

Where the code lives

Exercise	Package	Files
3 — Mocking, SmartHome	src\main\java\ise\mocking\exam_smarthome	SmartHome, Device, DeviceImpl, HomeOwner
	src\test\java\ise\mocking\exam_smarthome	SmartHomeTest.java (EasyMock), SmartHomeHandWrittenDoubleTest.java + DeviceMock.java
5 — SOLID, enrollment	src\main\java\ise\solid\exam_enrollment	EnrollmentService.java , PrerequisiteRule.java , CapacityRule, EnrollmentRule, Student, Course
	src\test\java\ise\solid\exam_enrollment	EnrollmentServiceTest.java — one @Nested block per task
4 — Black box	src\main\java\ise\blackbox\exam_gamelauncher	GameLauncher, AccessDecision
1 — Acceptance	src\main\java\ise\acceptance\exam_login	LoginService, LoginResult
Mutants for all four	src\test\java\ise\mutants\mockexam	MockExamMutants.java — 11 broken implementations

The SmartHome task has two solutions on purpose. The exam UML draws the test model as a hand-written `DeviceMock` class with a `lastCommand` field, but task 1 says *"add the test subject and mock attributes and annotate them"*, which is `@TestSubject` / `@Mock`. Whichever the grader expects, you have drilled it — and the contrast is itself quiz question 15.